Trial Task: Next.js WeChat & Alipay Integration

Overview

Build a small Next.js web app where a user can sign in with WeChat or Alipay and pay a fixed test amount with WeChat Pay or Alipay. Submit the code on GitHub with a sample `.env.example`, a README, and a short demo video.

The goal is to see how you read third-party API docs, handle OAuth and payment callbacks safely, and deliver a clean, runnable project. A working end-to-end flow matters more than visual polish.

Scope

In scope

- Login with WeChat (QR code on desktop)
- Login with Alipay
- Payment with WeChat Pay (Native / QR code on desktop)
- Payment with Alipay (PC website payment; mobile web payment is a bonus)
- Payment status page driven by the server-side callback, not the browser redirect
- A simple orders list for the signed-in user
- `.env.example`, README, demo video

Out of scope

- Refunds, subscriptions, partial payments
- Production deployment, custom domain, HTTPS certificates
- Mini Programs, native mobile apps, in-WeChat JSAPI payment (bonus only)
- Design system or pixel-perfect UI

Functional requirements

Pages

Route	Purpose
`/`	Landing page with “Login with WeChat” and “Login with Alipay” buttons
`/dashboard`	Protected. Shows user name, avatar, login provider, and a “Buy test item” card (e.g. 0.01 CNY)
`/checkout`	Choose WeChat Pay or Alipay, then show QR code (WeChat) or redirect (Alipay)
`/orders`	Protected. List of the user’s orders with status: pending, paid, failed, closed
`/orders/[id]`	Order detail; polls the server until status changes

Login

1. WeChat: OAuth 2.0 QR-code login via WeChat Open Platform (website application). Exchange `code` for access token server-side, fetch user info (openid / unionid, nickname, avatar).
2. Alipay: OAuth via Alipay Open Platform. Exchange `auth_code` for token server-side, fetch user info.
3. Validate the OAuth `state` parameter on every callback to prevent CSRF.
4. Create or update a local user record keyed by provider + provider user ID.
5. Keep the session in a secure, httpOnly cookie. Logout clears it.
6. A user can link both providers to one account (bonus).

Payment

1. Server creates the order first (amount, currency, provider, status = pending), then calls the provider.
2. WeChat Pay: use API v3 Native payment to get a `code_url`, render it as a QR code.
3. Alipay: use the PC website payment API and redirect the user to the Alipay checkout.
4. Handle the asynchronous payment notification (webhook) for both providers:
    - Verify the signature (WeChat Pay: decrypt resource with APIv3 key and verify platform signature; Alipay: verify RSA2 signature with Alipay public key).
    - Check that amount and order ID match the stored order.
    - Be idempotent: the same notification arriving twice must not double-update.
    - Respond in the exact format each provider expects.
5. Add an order-status query as a fallback if the notification is missed.
6. Never trust the amount or status coming from the browser.
7. Close or expire unpaid orders after 15 minutes (bonus).

Technical requirements

Stack

- Next.js (latest stable, App Router), TypeScript in strict mode
- Route Handlers (`app/api/...`) for OAuth callbacks and payment notifications
- Any database: SQLite + Prisma or Drizzle is fine; include migrations
- Auth: Auth.js custom providers or a hand-rolled session; explain the choice in the README
- Official SDKs or well-maintained libraries are allowed; list them in the README

Suggested structure

```
app/
  api/auth/wechat/callback/route.ts
  api/auth/alipay/callback/route.ts
  api/pay/wechat/create/route.ts
  api/pay/wechat/notify/route.ts
  api/pay/alipay/create/route.ts
  api/pay/alipay/notify/route.ts
  api/orders/[id]/route.ts
lib/
  wechat.ts   alipay.ts   session.ts   db.ts
```

Environment variables (`.env.example`)

```
# App
APP_URL=http://localhost:3000
SESSION_SECRET=replace-with-32-plus-random-chars
DATABASE_URL=file:./dev.db
PAYMENT_MODE=mock   # mock | sandbox | live

# WeChat Login (Open Platform website app)
WECHAT_APP_ID=
WECHAT_APP_SECRET=

# WeChat Pay (API v3)
WECHAT_PAY_MCH_ID=
WECHAT_PAY_APP_ID=
WECHAT_PAY_API_V3_KEY=
WECHAT_PAY_CERT_SERIAL_NO=
WECHAT_PAY_PRIVATE_KEY_PATH=./certs/apiclient_key.pem
WECHAT_PAY_PLATFORM_PUBLIC_KEY_PATH=./certs/wechatpay_public_key.pem
WECHAT_PAY_NOTIFY_URL=${APP_URL}/api/pay/wechat/notify

# Alipay (Login + Payment)
ALIPAY_APP_ID=
ALIPAY_PRIVATE_KEY=
ALIPAY_PUBLIC_KEY=
ALIPAY_GATEWAY=https://openapi-sandbox.dl.alipaydev.com/gateway.do
ALIPAY_NOTIFY_URL=${APP_URL}/api/pay/alipay/notify
ALIPAY_RETURN_URL=${APP_URL}/orders
```

Security rules

- No real secrets, keys or `.pem` files in the repo or git history. `.env*` (except `.env.example`) and `certs/` must be in `.gitignore`.
- All signing, token exchange and signature verification happen on the server only.
- Validate env vars at startup (e.g. with Zod) and fail with a clear message if one is missing.
- Log provider errors without logging secrets or full tokens.

Credentials, sandbox and mock mode

WeChat login and WeChat Pay need company-verified accounts, so the candidate cannot get real credentials alone. The app must therefore run fully in a mock mode, and switch to sandbox or live keys only through `.env`.

Integration	What the candidate can use	Who provides it
Alipay Login + Payment	Alipay Open Platform sandbox: sandbox app ID, keys, test buyer account	Candidate creates it with a personal Alipay developer account
WeChat Login	Verified Open Platform website app (AppID + AppSecret, authorised callback domain)	Company, if available; otherwise mock
WeChat Pay	Merchant ID, APIv3 key, merchant certificate, platform public key; test with 0.01 CNY	Company, if available; otherwise mock

Mock mode (`PAYMENT_MODE=mock`) is required:

- Mock login screens that return a fake WeChat / Alipay user through the same callback route
- Mock payment that generates a QR code / redirect and a “Simulate payment success” button
- The button must call the **same notify handler** with a correctly signed test payload, so the real verification logic is exercised
- Clearly labelled “MOCK MODE” banner in the UI

Callbacks on localhost: providers cannot reach `localhost`. Use a tunnel (ngrok or Cloudflare Tunnel) for sandbox testing and document the steps in the README.

Deliverables

1. GitHub repository (private, shared with the reviewer)

- Clean commit history with meaningful messages (no single “final commit”)
- `.env.example` with every variable, placeholder values only, and a one-line comment per variable
- `npm install && npm run dev` works in mock mode with no extra setup beyond copying `.env.example` to `.env.local`
- Lint and type-check pass (`npm run lint`, `npm run typecheck`)
- At least a few unit tests for signature verification and the notify handler (idempotency)

2. README.md

- Setup steps for mock, sandbox and live modes
- How to get Alipay sandbox keys and where to put them
- How to expose callbacks with a tunnel
- Sequence of the login flow and payment flow (a simple diagram or numbered list)
- Known limitations and what you would do next

3. Demo video (3–5 minutes, screen recording with voice-over, MP4 or unlisted link)

1. Start the app from a fresh clone
2. Login with WeChat and with Alipay (real sandbox or mock, state which)
3. Create an order and pay with Alipay sandbox using the sandbox buyer account
4. Create an order and pay with WeChat Pay (real 0.01 CNY or mock)
5. Show the order status changing to paid via the notification, and the database record

6. Show what happens on a failed or cancelled payment
7. Walk through the notify handler code for 30–60 seconds

The video must not show any real secret, key or full `.env` file.

Acceptance criteria and scoring

The task passes when all must-haves work in mock mode and Alipay works in sandbox. Scores below are out of 100.

Area	What we check	Points
Login	Both providers, `state` validated, session cookie secure, logout works	20
Payment flow	Server-created orders, QR / redirect, status page updates from notify	25
Security	Signature verification, amount check, idempotency, no secrets in repo	20
Code quality	TypeScript, structure, error handling, tests	15
Documentation	README, `.env.example`, clear setup in under 10 minutes	10
Demo video	Covers every step in the script, clear explanation	10

Automatic fail: real secrets committed (even if later deleted), payment marked as paid from the browser redirect alone, or the project does not start from a fresh clone.

Bonus (up to +10): WeChat in-app browser login and JSAPI payment, Alipay mobile web payment, order expiry, account linking, Docker setup.

Timeline and submission

Double-check the deadline and submission process with HR before you start. The plan below is a suggested pace for 5 working days.

Day	Checkpoint
1	Project setup, mock mode skeleton, short plan sent to manager
2	Alipay login + payment working in sandbox
3	WeChat login + payment (real or mock), notify handlers with signature checks
4	Orders page, tests, README
5	Demo video, final review, submit

Submit as HR directs, including: the GitHub repo link, the video link, and a 3–5 line note on what works, what is mocked, and what you would improve.